

Evasion Attacks

A. M. Sadeghzadeh, Ph.D.

Sharif University of Technology
Computer Engineering Department (CE)
Trustworthy and Secure AI Lab



November 3, 2026

Today's Agenda

1 Recap

2 Type of Adversarial Attacks

3 Fast Gradient Sign Method(FGSM) Attack

4 Projected Gradient Descent

Recap

Adversarial Examples



Figure: Adversarial examples generated for AlexNet.(Left) is a correctly predicted sample, (center) difference between correct image, and image predicted incorrectly magnified by 10x (values shifted by 128 and clamped), (right) adversarial example. All images in the right column are predicted to be an *"ostrich, Struthio camelus"*. Average distortion based on 64 examples is 0.006508.

Type of Adversarial Attacks

Threat Models

White-Box

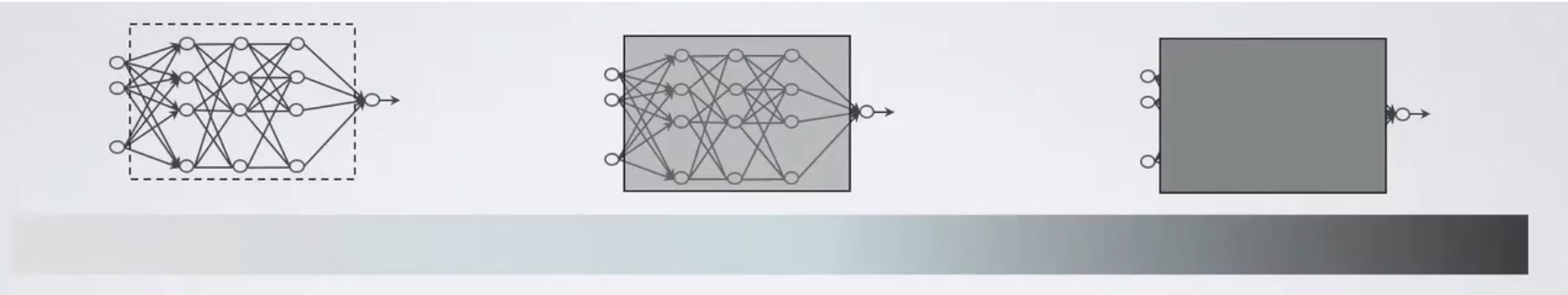
- The adversary has complete access to the algorithm, architecture, parameters, hyper-parameters, and input-output type of the target model.
- Introduced L-BFGS attack has white-box threat model.

Gray-Box

- The adversary has partial access to the algorithm, architecture, parameters, hyper-parameters, and input-output type of the target model.

Black-Box

- The adversary only has API access to the target model.



Targeted and Untargeted Adversarial Examples

Untargetted attack

- The adversary wants to change the predication of the classifier to a wrong class.
  - Example: an image of a “cat” classified as “dog” or “car” (any wrong label).

Targeted attack

- The adversary wants to change the predication of the classifier to a given target class.
  - Example: an image of a “cat” deliberately modified to be classified as “airplane.”

L_P Norm

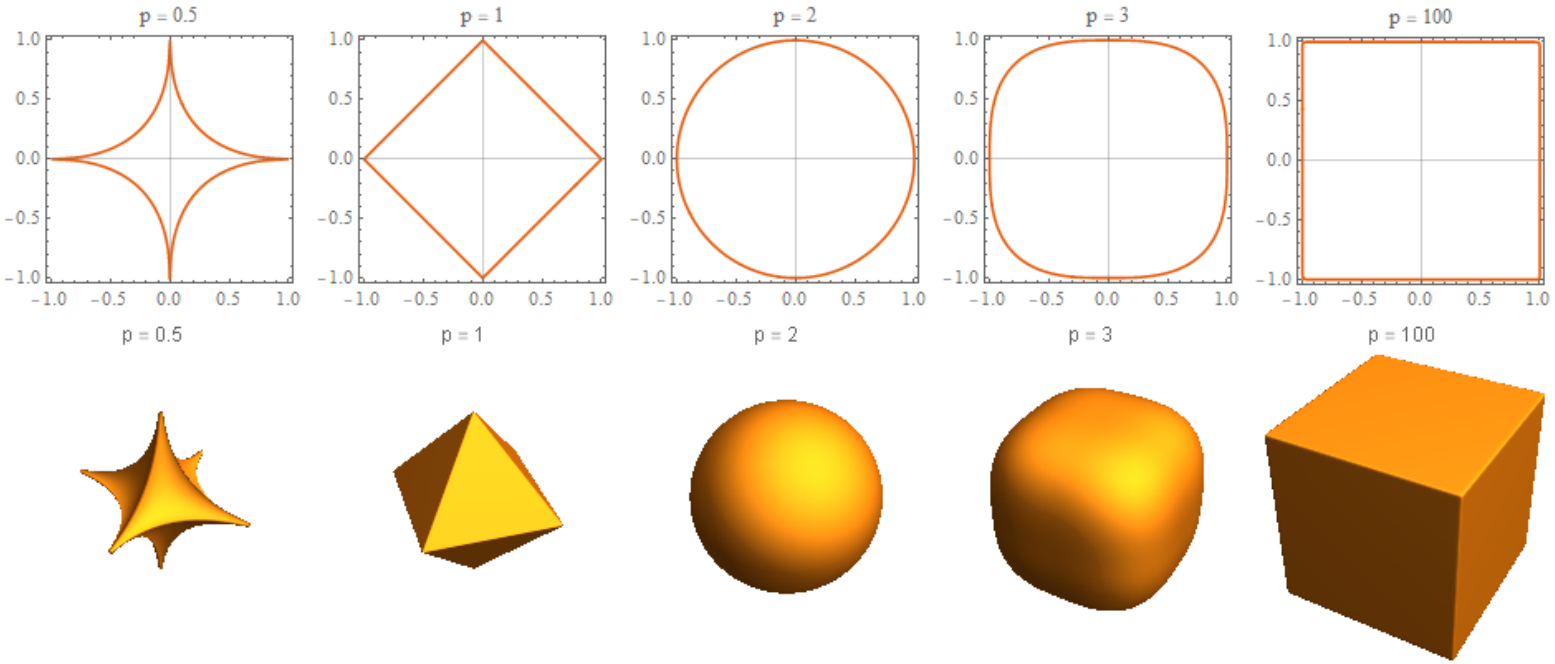
Let $p \geq 1$ be a real number, the P-norm (also called L_P-norm) of vector $\mathbf{x} = [x_1, x_2, \ldots, x_d]^T$ is

$$\|\boldsymbol{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}$$

L_P Norm

Let $p \geq 1$ be a real number, the P-norm (also called L_P-norm) of vector $\mathbf{x} = [x_1, x_2, \ldots, x_d]^T$ is

$$\|\boldsymbol{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}$$



The boundary of $\|\boldsymbol{x}\|_P = 1$
(Source).

Is L_P with $P < 1$ really a norm? The answer is no, because it violates the triangle inequality (See Convex Optimization by Stephen Boyd)

L_P Norm

The L_P distance is written $\|x - x'\|_P$, where $x, x' \in \mathbb{R}^n$ and the P-norm $\|.\|_P$ is defined as

$$\|x - x'\|_p = \left(\sum_{i=1}^{n} |x_i - x'_i|^p \right)^{1/p}$$

L_P Norm

The L_P distance is written $\|x-x'\|_P$, where $x, x' \in \mathbb{R}^n$ and the P-norm $\|.\|_P$ is defined as

$$\|x - x'\|_p = \left(\sum_{i=1}^{n} |x_i - x'_i|^p \right)^{1/p}$$

- **L_0 distance** measures the number of coordinates i such that $x_i \neq x'_i$.
  - Thus, the L_0 distance corresponds to the number of pixels that have been altered in an image.

L_P Norm

The L_P distance is written $\|x - x'\|_P$, where $x, x' \in \mathbb{R}^n$ and the P-norm $\|.\|_P$ is defined as

$$\|x - x'\|_p = \left(\sum_{i=1}^{n} |x_i - x'_i|^p \right)^{1/p}$$

- **L_0 distance** measures the number of coordinates i such that $x_i \neq x'_i$.
  - Thus, the L_0 distance corresponds to the number of pixels that have been altered in an image.

- **L_2 distance** measures the standard Euclidean (root mean-square) distance between x and x'.
  - The L_2 distance can remain small when there are many small changes to many pixels.

L_P Norm

The L_P distance is written $\|x-x'\|_P$, where $x, x' \in \mathbb{R}^n$ and the P-norm $\|.\|_P$ is defined as

$$\|x - x'\|_p = \left(\sum_{i=1}^{n} |x_i - x'_i|^p \right)^{1/p}$$

- $\boldsymbol{L_0}$ **distance** measures the number of coordinates i such that $x_i \neq x'_i$.
  - Thus, the L_0 distance corresponds to the number of pixels that have been altered in an image.

- $\boldsymbol{L_2}$ **distance** measures the standard Euclidean (root mean-square) distance between x and x'.
  - The L_2 distance can remain small when there are many small changes to many pixels.

- $\boldsymbol{L_\infty}$ **distance** measures the maximum change to any of the coordinates

$$\|x - x'\|_\infty = max(|x_1 - x'_1|, ..., |x_n - x'_n|).$$

  - For images, we can imagine there is a maximum budget, and each pixel is allowed to be changed by up to this limit, with no limit on the number of pixels that are modified.

Fast Gradient Sign Method(FGSM) Attack

Explaining and Harnessing Adversarial Examples

Published as a conference paper at ICLR 2015

EXPLAINING AND HARNESSING ADVERSARIAL EXAMPLES

Ian J. Goodfellow, Jonathon Shlens & Christian Szegedy
Google Inc., Mountain View, CA
{goodfellow,shlens,szegedy}@google.com

Abstract

- The primary cause of neural networks' vulnerability to adversarial perturbation is their **linear nature**.
- Giving the first explanation of the most intriguing fact about them: their **generalization** across architectures and training sets.
- In this lecture, we introduce a **simple and fast method of generating adversarial examples**.

Smoothness Prior with L_∞

- For problems with well-separated classes, we expect the classifier to assign the same class to x and $x' = x + \eta$ so long as $\|\eta\|_\infty \leq \epsilon$, where ϵ is small.
  - For $\mathbf{x} = [x_1, x_2, \ldots, x_d]^T$, $\|\mathbf{x}\|_\infty = \max_i |x_i|$.

The Linear Explanation of Adversarial Examples

- Let $\hat{y} = \boldsymbol{w}^T \boldsymbol{x}$ and $\boldsymbol{x}' = \boldsymbol{x} + \eta$, the dot product between weight vector $\boldsymbol{w}$ and adversarial example x'

The Linear Explanation of Adversarial Examples

- Let $\hat{y} = \boldsymbol{w}^T \boldsymbol{x}$ and $\boldsymbol{x}' = \boldsymbol{x} + \eta$, the dot product between weight vector $\boldsymbol{w}$ and adversarial example x' is as follows

$$\hat{y}' = \boldsymbol{w}^T \boldsymbol{x}' = \boldsymbol{w}^T (\boldsymbol{x} + \eta) = \boldsymbol{w}^T \boldsymbol{x} + \boldsymbol{w}^T \eta \Rightarrow \hat{y}' - \hat{y} = \boldsymbol{w}^T \eta$$

  The adversarial perturbation causes the activation to grow by $\boldsymbol{w}^T \eta$.

The Linear Explanation of Adversarial Examples

- Let $\hat{y} = \boldsymbol{w}^T \boldsymbol{x}$ and $\boldsymbol{x}' = \boldsymbol{x} + \eta$, the dot product between weight vector $\boldsymbol{w}$ and adversarial example x' is as follows

$$\hat{y}' = \boldsymbol{w}^T \boldsymbol{x}' = \boldsymbol{w}^T (\boldsymbol{x} + \eta) = \boldsymbol{w}^T \boldsymbol{x} + \boldsymbol{w}^T \eta \Rightarrow \hat{y}' - \hat{y} = \boldsymbol{w}^T \eta$$

  The adversarial perturbation causes the activation to grow by $\boldsymbol{w}^T \eta$.
- To generate adversarial example for x, we should maximize $\boldsymbol{w}^T \eta$, such that $\|\eta\|_\infty \leq \epsilon$. Therefore, we have the following maximization problem.

$$\begin{aligned} &\underset{\eta}{\text{argmax}} < \boldsymbol{w}, \eta > \\ &s.t. \quad \|\eta\|_\infty \leq \epsilon \end{aligned}$$

The Linear Explanation of Adversarial Examples

- Let $\hat{y} = \boldsymbol{w}^T \boldsymbol{x}$ and $\boldsymbol{x}' = \boldsymbol{x} + \eta$, the dot product between weight vector $\boldsymbol{w}$ and adversarial example x' is as follows

$$\hat{y}' = \boldsymbol{w}^T \boldsymbol{x}' = \boldsymbol{w}^T (\boldsymbol{x} + \eta) = \boldsymbol{w}^T \boldsymbol{x} + \boldsymbol{w}^T \eta \Rightarrow \hat{y}' - \hat{y} = \boldsymbol{w}^T \eta$$

  The adversarial perturbation causes the activation to grow by $\boldsymbol{w}^T \eta$.
- To generate adversarial example for x, we should maximize $\boldsymbol{w}^T \eta$, such that $\|\eta\|_\infty \leq \epsilon$. Therefore, we have the following maximization problem.

$$\begin{aligned} &\underset{\eta}{\text{argmax}} < \boldsymbol{w}, \eta > \\ &s.t. \quad \|\eta\|_\infty \leq \epsilon \end{aligned}$$

  The solution to the above problem is $\eta^* = \epsilon.\text{sign}(\boldsymbol{w})$, we have

$$\hat{y}' - \hat{y} = \boldsymbol{w}^T \eta^* = \boldsymbol{w}^T \epsilon.\text{sign}(\boldsymbol{w}) = \epsilon \|\boldsymbol{w}\|_1$$

The Linear Explanation of Adversarial Examples

- Let $\hat{y} = \boldsymbol{w}^T \boldsymbol{x}$ and $\boldsymbol{x}' = \boldsymbol{x} + \eta$, the dot product between weight vector $\boldsymbol{w}$ and adversarial example x' is as follows

$$\hat{y}' = \boldsymbol{w}^T \boldsymbol{x}' = \boldsymbol{w}^T (\boldsymbol{x} + \eta) = \boldsymbol{w}^T \boldsymbol{x} + \boldsymbol{w}^T \eta \Rightarrow \hat{y}' - \hat{y} = \boldsymbol{w}^T \eta$$

  The adversarial perturbation causes the activation to grow by $\boldsymbol{w}^T \eta$.
- To generate adversarial example for x, we should maximize $\boldsymbol{w}^T \eta$, such that $\|\eta\|_\infty \leq \epsilon$. Therefore, we have the following maximization problem.

$$\begin{aligned} &\underset{\eta}{\text{argmax}} < \boldsymbol{w}, \eta > \\ &s.t. \quad \|\eta\|_\infty \leq \epsilon \end{aligned}$$

  The solution to the above problem is $\eta^* = \epsilon.\text{sign}(\boldsymbol{w})$, we have

$$\hat{y}' - \hat{y} = \boldsymbol{w}^T \eta^* = \boldsymbol{w}^T \epsilon.\text{sign}(\boldsymbol{w}) = \epsilon \|\boldsymbol{w}\|_1$$

- If $\boldsymbol{w}$ has n dimensions and the average magnitude of an element of the weight vector is m, then the **activation will grow by ϵmn.** Thereby, as the dimension of the input increases, the value of $\hat{y}' - \hat{y}$ will grow.
- This explanation shows that a simple **linear model can have adversarial examples** if its input has **sufficient dimensionality**.

Linear Perturbation for Non-linear Models

The **linear view** of adversarial examples suggests a **fast** way of generating them.

- It is hypothesized that deep nets are **too linear** to resist adversarial perturbations (ReLU activation function).
- More nonlinear models such as **sigmoid or tanh** networks are carefully tuned to spend most of their time in the **non-saturating, more linear regime**.

Hence, we suppose Deep nets have **linear behavior** in the **vicinity** of each data point.

Linear Perturbation for Non-linear Models

The **linear view** of adversarial examples suggests a **fast** way of generating them.

- It is hypothesized that deep nets are **too linear** to resist adversarial perturbations (ReLU activation function).
- More nonlinear models such as **sigmoid or tanh** networks are carefully tuned to spend most of their time in the **non-saturating, more linear regime**.

Hence, we suppose Deep nets have **linear behavior** in the **vicinity** of each data point.

Recall: Taylor Series (Expansion)

Suppose n is a positive integer and $f : \mathbb{R} \to \mathbb{R}$ is n times differentiable at a point x_0. Then

$$f(x) = \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!}(x - x_0)^k + R_n(x, x_0)$$

$$= f(x_0) + f'(x_0)(x - x_0) + \frac{f"(x_0)}{2}(x - x_0)^2 + \ldots$$

where the remainder R_n satisfies

$$R_n(x, x_0) = o(|x - x_0|^n) \text{ as } x \to x_0.$$

Definition: A sequence of numbers X_n is said to be $o(r_n)$ if $\frac{X_n}{r_n} \to 0$ as $n \to \infty$.

Linear Perturbation for Non-linear Models

The **linear view** of adversarial examples suggests a **fast** way of generating them.

- It is hypothesized that deep nets are **too linear** to resist adversarial perturbations (ReLU activation function).
- More nonlinear models such as **sigmoid or tanh** networks are carefully tuned to spend most of their time in the **non-saturating, more linear regime**.

Hence, we suppose Deep nets have **linear behavior** in the **vicinity** of each data point.

Consequently, we can linearly approximate classifier $f : \mathbb{R}^d \to \mathbb{R}$ around data point x_0 by **Taylor expansion**. We have:

$$f(x) = f(x_0) + (x - x_0)^T \nabla_x f(x)$$

Linear Perturbation for Non-linear Models

The **linear view** of adversarial examples suggests a **fast** way of generating them.

- It is hypothesized that deep nets are **too linear** to resist adversarial perturbations (ReLU activation function).
- More nonlinear models such as **sigmoid or tanh** networks are carefully tuned to spend most of their time in the **non-saturating, more linear regime**.

Hence, we suppose Deep nets have **linear behavior** in the **vicinity** of each data point.

Consequently, we can linearly approximate classifier $f : \mathbb{R}^d \to \mathbb{R}$ around data point x_0 by **Taylor expansion**. We have:

$$f(x) = f(x_0) + (x - x_0)^T \nabla_x f(x)$$

Let $x' = x_0 + \eta$, we get

$$f(x') = f(x + \eta) = f(x_0) + (\eta)^T \nabla_x f(x) \Rightarrow f(x') - f(x_0) = (\eta)^T \nabla_x f(x)$$

To maximize difference between $f(x)$ and $f(x')$, we should maximize $< \eta^T, \nabla_x f(x) >$. Given $\|\eta\|_\infty \leq \epsilon$, we have

$$\eta = \epsilon . sign(\nabla_x f(x))$$

Linear Perturbation for Non-linear Models

The **linear view** of adversarial examples suggests a **fast** way of generating them.

- It is hypothesized that deep nets are **too linear** to resist adversarial perturbations (ReLU activation function).
- More nonlinear models such as **sigmoid or tanh** networks are carefully tuned to spend most of their time in the **non-saturating, more linear regime**.

Hence, we suppose Deep nets have **linear behavior** in the **vicinity** of each data point.

Consequently, we can linearly approximate classifier $f : \mathbb{R}^d \rightarrow \mathbb{R}$ around data point x_0 by **Taylor expansion**. We have:

$$f(x) = f(x_0) + (x - x_0)^T \nabla_x f(x)$$

Let $x' = x_0 + \eta$, we get

$$f(x') = f(x + \eta) = f(x_0) + (\eta)^T \nabla_x f(x) \Rightarrow f(x') - f(x_0) = (\eta)^T \nabla_x f(x)$$

To maximize difference between $f(x)$ and $f(x')$, we should maximize $< \eta^T, \nabla_x f(x) >$. Given $\|\eta\|_\infty \leq \epsilon$, we have

$$\eta = \epsilon.sign(\nabla_x f(x))$$

We can replace classifier output with cost function J

$$\eta = \epsilon.sign(\nabla_x J(\theta, x, y))$$

Fast Gradient Sign Method (FGSM)

Let $\boldsymbol{\theta}$ be the parameters of a model, $\boldsymbol{x}$ the input to the model, y the label associated with $\boldsymbol{x}$ and $J(\boldsymbol{\theta}, \boldsymbol{x}, y)$ be the cost used to train the neural network.

We can linearize the cost function around the current value of $\boldsymbol{\theta}$, obtaining an optimal max-norm constrained perturbation of

$$\boldsymbol{\eta} = \epsilon \, sign(\nabla_x J(\boldsymbol{\theta}, \boldsymbol{x}, y))$$

We refer to this as the "**fast gradient sign method**" of generating adversarial examples.

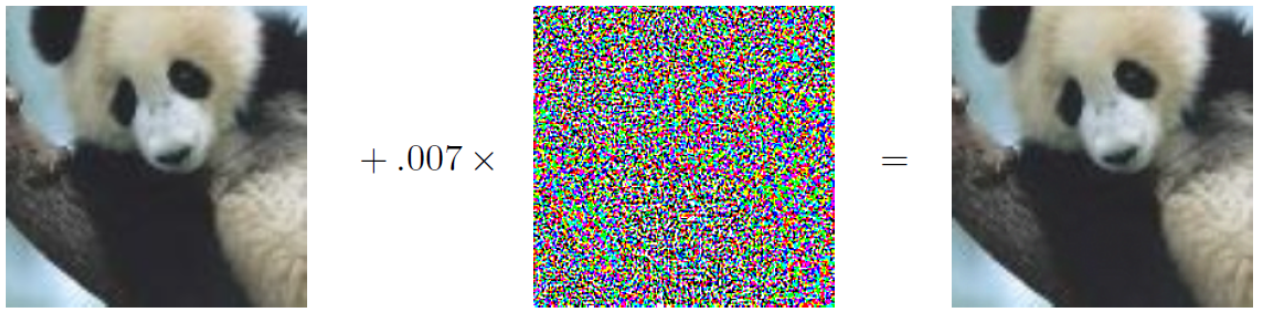


Figure 1: A demonstration of fast adversarial example generation applied to GoogLeNet (Szegedy et al., 2014a) on ImageNet. By adding an imperceptibly small vector whose elements are equal to the sign of the elements of the gradient of the cost function with respect to the input, we can change GoogLeNet's classification of the image. Here our ϵ of .007 corresponds to the magnitude of the smallest bit of an 8 bit image encoding after GoogLeNet's conversion to real numbers.

Targeted and Untargeted FGSM

Untargetted attack

- The adversary wants to change the predication of the classifier to a wrong class.
  - Untargeted FGSM attack on clean data $(\boldsymbol{x}, y)$

$$\boldsymbol{x}_{adv} = \boldsymbol{x} + \epsilon.sign(\nabla_x J(\boldsymbol{\theta}, \boldsymbol{x}, y))$$

Targeted attack

- The adversary wants to change the predication of the classifier to a given target class.
  - Targeted FGSM attack on clean data $(\boldsymbol{x}, y)$ for given target class t

$$\boldsymbol{x}_{adv} = \boldsymbol{x} - \epsilon.sign(\nabla_x J(\boldsymbol{\theta}, \boldsymbol{x}, t))$$

Observations

- Adversarial examples can be explained as a property of high-dimensional dot products. They are a result of models being **too linear, rather than too nonlinear**.
- The **direction of perturbation**, rather than the specific point in space, matters most.
- Because it is the direction that matters most, adversarial perturbations **generalize** across different clean examples.

Potemkin village - Clever Hans

- These results suggest that classifiers based on modern machine learning techniques, even those that obtain excellent performance on the test set, are not learning the true underlying concepts that determine the correct output label.
- Instead, these algorithms have built a **Potemkin village that works well on naturally occuring data, but is exposed as a fake when one visits points in space that do not have high probability in the data distribution.**
- Clever Hans was a horse that was claimed to have performed arithmetic and other intellectual tasks. After a formal investigation in 1907, psychologist Oskar Pfungst demonstrated that the horse was not actually performing these mental tasks, but was watching the reactions of his trainer.





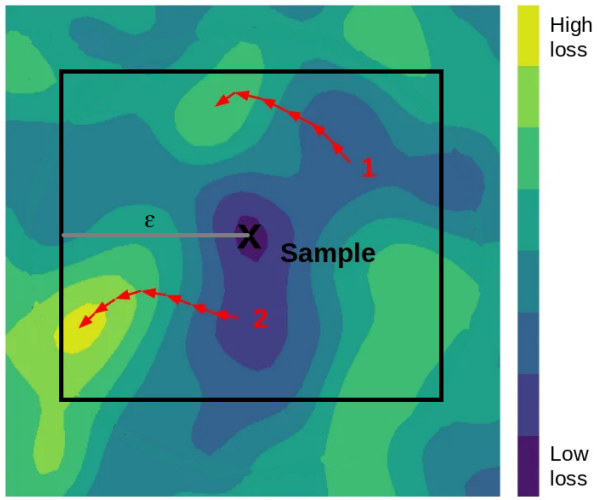
Projected Gradient Descent

Projected Gradient Descent (PGD) Attack

A more powerful adversary is the **multi-step variant**, which is essentially projected gradient descent (PGD) on the negative loss function:

$$
\begin{aligned}
x^{(0)} &= \text{clip}_{[0,1]}\{x + \mathcal{U}(-\epsilon, +\epsilon)\}, \\
\delta^{(t+1)} &= \alpha \cdot \text{sign}\left(\nabla_x \mathcal{L}(f_\theta(x^{(t)}), y)\right), \\
x^{(t+1)} &= \text{clip}_{[\max(0,x-\epsilon),\min(1,x+\epsilon)]}\{x^{(t)} + \delta^{(t+1)}\}
\end{aligned}
$$

where x is a natural data, $\mathcal{U}$ is uniform distribution, $\text{clip}_{[a,b]}\{x\}$ function is used to trim values outside interval $[a, b]$ to the interval edges, ϵ is the radius of allowed perturbation $\|\cdot\|_\infty \leq \epsilon$, and t is iteration index.



Source

PGD Algorithm

Algorithm 1 Projected gradient descent.

Input: clean data x, true labels y, model f
Parameters: perturbation budget ϵ, step size α, random initialization `Random Noise`, number of steps k
if `Random Noise` **then**
    $x^{(0)} \leftarrow x + \mathcal{U}(-\epsilon, +\epsilon)$
    $x^{(0)} \leftarrow \text{clip}_{[0,1]}\{x^{(0)}\}$ ▷ Ensure valid pixel range
else
    $x^{(0)} \leftarrow x$
end if
for $t = 1, \ldots, k$ **do**
    $\delta^{(t)} \leftarrow \text{sign}\left(\nabla \mathcal{L}\left(f(x^{(t-1)}, y)\right)\right)$
    $x^{(t)} \leftarrow x^{(t-1)} + \alpha \cdot \delta^{(t)}$
    $x^{(t)} \leftarrow \text{clip}_{[x-\epsilon, x+\epsilon]}\{x^{(t)}\}$ ▷ Enforce perturbation budget
    $x^{(t)} \leftarrow \text{clip}_{[0,1]}\{x^{(t)}\}$ ▷ Ensure valid pixel range
end for

The Landscape of Adversarial Examples

- While there are many local maxima spread widely apart within $x_i + \mathcal{S}$, they tend to have **very well-concentrated loss values**.
- This echoes the folklore belief that training neural networks is possible because the loss (as a function of model parameters) typically has many local minima with very similar values.

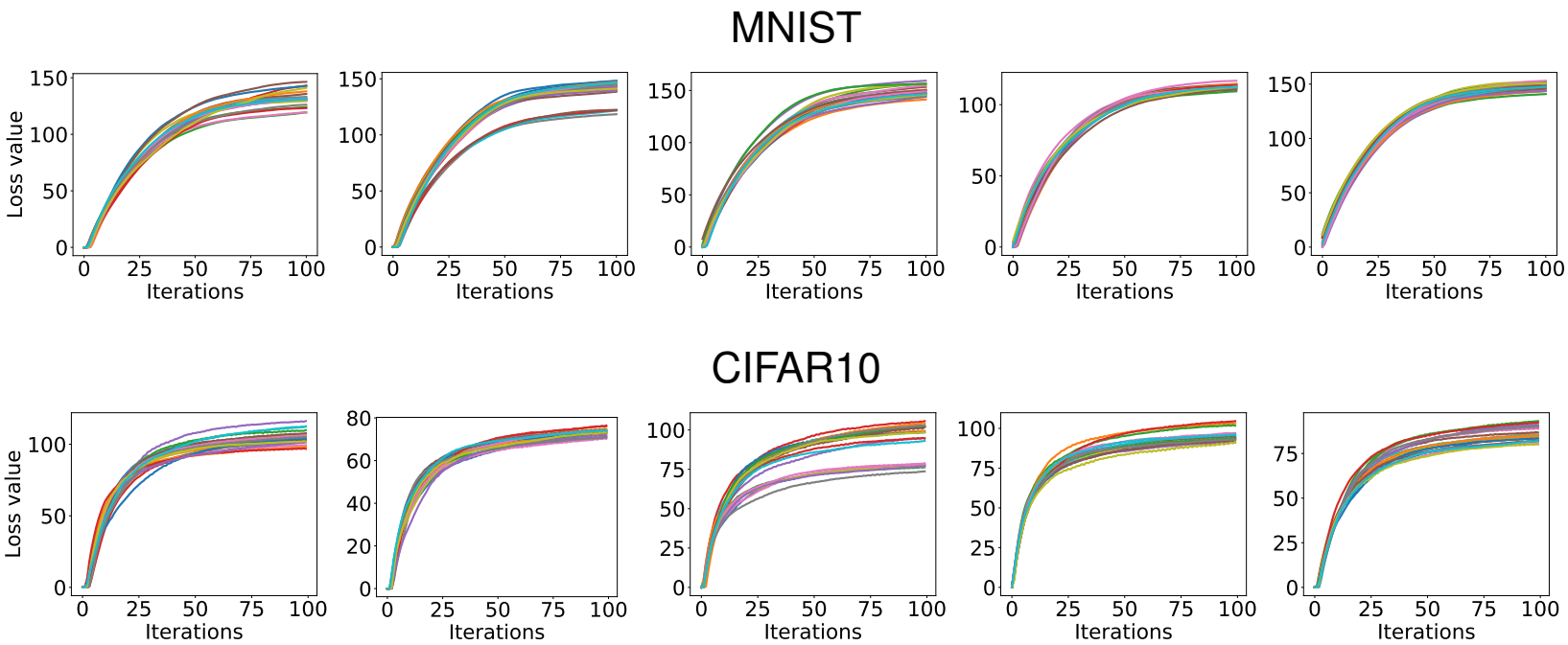


Figure: Loss function value over PGD iterations for 20 random restarts on random examples.

First-Order Adversaries

The concentration phenomenon suggests an intriguing view on the problem in which robustness against the PGD adversary yields robustness against all first-order adversaries:

- As long as the adversary only uses gradients of the loss function with respect to the input, we conjecture that it will not find significantly better local maxima than PGD.
- Of course, our exploration with PGD does not preclude the existence of some isolated maxima with much larger function value.
- Experiments suggest that such better local maxima are hard to find with first order methods.

References

- C. Szegedy, W. Zaremba, I. Sutskever, et al., "Intriguing properties of neural networks," in 2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014.
- I. Goodfellow, J. Shlens, C. Szegedy, "Explaining and Harnessing Adversarial Examples" in 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015.
- D. Baehrens, T. Schroeter, S. Harmeling, et al., "How to explain individual classification decisions." The Journal of Machine Learning Research 11 (2010): 1803-1831.